



Institute of Engineering and Technology (IET)

Major Project Synopsis (PR1107)

SwarmSec: An Autonomous Multi-Agent System for Cloud Security

PREPARED BY

Sarthak Pandit (2023BTech072)

Tushar Parihar (2023BTech093)

FACULTY GUIDE

Mr. Gaurav Raj

July, 2026

Contents

1	Abstract	4
2	Introduction	5
3	Literature Review	6
3.1	LLM-Based Security Agents.....	6
3.2	Graph-Based Network Modelling	6
3.3	Traditional Detection and Response Tools.....	6
3.4	Gap in Existing Work.....	6
4	Problem Statement	8
5	Objectives	9
6	Scope of Work	10
7	Technology Stack	11
8	System Design	12
8.1	Design Philosophy	12
8.2	Agent Roles.....	12
8.3	Data Flow	12
8.4	Deployment View	13
9	Methodology	14
10	Threat Scenarios and Comparison with Traditional Security Tools	15
10.1	Scenario 1: Multi-Stage Port Scanning and Lateral Movement.....	15
10.2	Scenario 2: Layer-7 API Abuse	15
10.3	Scenario 3: Malicious Routing and IP/ARP Spoofing	15
10.4	Scenario 4: Container Reverse-Shell Execution.....	15
10.5	Comparison Summary.....	15
11	Fault Tolerance and Reliability	17
12	Work Plan	18
13	Expected Outcomes	19
14	References	20

List of Figures

8.1	SwarmSec agent pipeline and data flow	13
-----	---	----

List of Tables

7.1	Technology stack used in SwarmSec	11
10.1	SwarmSec vs. traditional security tools	16
12.1	Week-wise work plan	18

1. Abstract

Modern cloud infrastructure runs on containers, microservices, and interconnected virtual machines that can be spun up or torn down within seconds. This flexibility is what makes cloud systems attractive for businesses, but it also makes them harder to secure, because an attacker no longer needs to break into one big server – they only need to compromise one small, poorly monitored container and then move sideways through the network from there. Most of the attacks that cause real damage today, such as lateral movement after a breach, low-and-slow API abuse, route spoofing, and reverse-shell execution inside a container, unfold as a sequence of small steps rather than one obvious event. Traditional security tools such as firewalls, signature-based intrusion detection systems, and SIEM/SOAR platforms are good at catching attacks that match a known pattern, but they struggle badly with anything that deviates even slightly from a pre-written rule, and they still depend on a human engineer to read the alert and manually apply a fix.

This project proposes SwarmSec, an autonomous multi-agent security system that is designed to close this gap. Instead of a single monolithic detection script, SwarmSec is built as a pipeline of cooperating agents: agents that monitor logs and traffic, an agent that reasons about whether an event is actually malicious using a combination of machine learning and LLM-based log analysis, an agent that checks the incident against a live graph of the network topology stored in Neo4j to estimate how far the damage could spread, an agent that writes a targeted fix specific to that incident, and an agent that tests the fix inside an isolated Docker sandbox before it is ever allowed to touch the live system. Every step of this process is logged so that the entire incident, from detection to resolution, can be reviewed later.

The system is scoped to four concrete attack scenarios – multi-stage port scanning and lateral movement, Layer-7 API abuse, malicious routing and IP spoofing, and container reverse-shell execution – and its performance is compared against a baseline made of standard firewall and Suricata rules on the same simulated attacks. The expected result is a working prototype that detects and safely resolves these incidents faster than a traditional rule-based setup, while keeping a complete, auditable record of every action it takes.

2. Introduction

Over the last decade, the shift from monolithic applications running on a handful of physical servers to distributed, container-based deployments running across dozens or hundreds of virtual machines has completely changed what a security incident looks like. In a traditional setup, a security team could reasonably assume that most of the important traffic passed through a small number of chokepoints, and a firewall sitting at those chokepoints was often enough to catch most attacks. In a modern cloud-native setup, traffic moves constantly between containers, services talk to each other over internal APIs, and a single compromised component can reach dozens of other components before anyone notices something is wrong.

This shift has not been matched by an equivalent shift in how security tools work. Firewalls, network-based intrusion detection systems such as Snort and Suricata, and SIEM/SOAR platforms such as Splunk are still, at their core, signature and threshold-based systems. They compare incoming traffic or log entries against a library of known-bad patterns, and they raise an alert – or in more advanced SOAR setups, run a fixed automation script – when a match is found. This works well for attacks that have been seen before, but it fails quietly when an attacker changes even one small detail of their approach, because the new traffic simply does not match any existing rule. On top of this, most automation in this space is still built around rigid if-else playbooks that are written once and rarely updated, and any automated script that is different from what was expected typically stops and waits for a human to step in.

Recent research on large language models has opened up a different way of approaching this problem. Instead of matching fixed signatures, an LLM can read raw, messy log data and reason about what it actually means, in much the same way a human security analyst would. Work such as CyberLLM has shown that this reasoning can be organised into a team of specialised agents working under a safety layer, rather than one large model trying to do everything at once, and other research has simulated LLM agents collaborating during real incident-response exercises with encouraging results. Separately, the security industry has increasingly turned to graph databases such as Neo4j to model network topology, because a graph naturally represents how one compromised node can reach another, which is exactly the kind of blast-radius question a defender needs answered quickly during an incident.

SwarmSec is an attempt to bring these two ideas together and apply them specifically to infrastructure-level attacks inside a cloud environment. Rather than only alerting a human or running a fixed script, SwarmSec investigates an incident, checks its position in the network graph, writes a fix tailored to that specific incident, tests the fix safely before deploying it, and keeps a full record of what it did and why. The rest of this document lays out the problem this project is solving, the objectives and scope of the work, the technology used, the system design, the week-by-week plan, and the outcomes expected by the end of the project.

3. Literature Review

3.1 LLM-Based Security Agents

The idea of using large language models as autonomous or semi-autonomous security agents has picked up pace only in the last two to three years. CyberLLM [1] proposes a multi-agent, LLM-orchestrated framework for automotive software that combines a deterministic detection layer (regex rules, static analysis, topology checks) with an LLM refinement pass, and then routes any proposed fix through a formal safety guard before it is allowed to execute. This two-layer idea – a fast, rule-based first pass followed by a slower, LLM-based reasoning pass – is directly relevant to SwarmSec’s detection agent design. Separately, work on multi-agent collaboration in incident response [2] has used tabletop cybersecurity exercises to compare how centralised, decentralised, and hybrid teams of LLM agents perform during a simulated breach, and found that splitting responsibilities across specialised agents generally produced better outcomes than a single generalist agent trying to do everything.

3.2 Graph-Based Network Modelling

Several papers have looked at using graph databases, and Neo4j specifically, to represent network topology and attack paths. Neo4j’s own cybersecurity use-case documentation [3] describes “blast radius” analysis – instantly visualising everything an attacker could reach once they land on one compromised machine – as one of the main reasons security teams adopt a graph model instead of a flat, siloed log view. Academic work on graph-database-based network security situation awareness [4] demonstrated that storing network topology, vulnerability, and personnel data together in a graph database made it noticeably faster to query multi-hop relationships compared to a traditional relational database, since graph traversal in Neo4j follows direct edges instead of expensive join operations. Similar work on dynamic attack graphs in IoT environments [5] used Neo4j and its Cypher query language to model how an attack could propagate as the network itself changed over time, which is close to the kind of live topology awareness SwarmSec’s graph-context agent needs.

3.3 Traditional Detection and Response Tools

Signature-based systems such as Suricata [7] remain the industry standard for network intrusion detection and prevention. They are fast, well-understood, and easy to audit, but they are explicitly rule-driven – every new attack pattern needs a new signature to be written and deployed before the tool can catch it, and there is no built-in way for the tool to reason about traffic it has never seen before.

3.4 Gap in Existing Work

Taken individually, none of this prior work fully solves the problem SwarmSec is aimed at. LLM-based agent research shows that reasoning about an incident is possible, graph-database research shows that network context can be modelled and queried efficiently, and traditional IDS/SOAR tools show what a production-grade detection pipeline looks like today. What is

missing is a system that combines LLM-based reasoning, graph-based network context, and a safe pre-deployment testing step into a single working pipeline aimed specifically at container and infrastructure-level attacks. This is the gap SwarmSec is intended to fill.

4. Problem Statement

The core problem this project addresses can be broken down into three related issues that show up again and again in cloud security operations.

Reactive, signature-bound detection. Most detection tools only recognise an attack once it matches a rule that was written in advance. If an attacker scans ports slowly enough to stay under a rate threshold, or sends API requests that look individually normal but form an unusual pattern over time, a signature-based tool has no way of flagging it, because there is no single request or packet that looks wrong on its own. The attack only becomes visible when someone looks at the sequence of events as a whole, which is exactly what static rules are not built to do.

No awareness of network context. Even when an alert is raised correctly, most tools treat it in isolation. A security engineer looking at an alert from one container has no automatic way of knowing which other services that container can reach, what data it has access to, or how serious the alert really is in the context of the wider system. This forces engineers to manually piece together the network layout during an active incident, which wastes time exactly when time matters most.

Risky, human-dependent remediation. Automated response scripts that do exist are usually written once, hardcoded, and run directly against production. If the script does not match the situation perfectly, it can fail outright, or worse, take an action that causes unplanned downtime. Because of this risk, many organisations keep a human in the loop for every remediation step, which means the actual time between an alert being raised and a fix being applied can stretch from a few minutes to several hours, depending on when an on-call engineer is available.

Together, these three issues mean that the tools organisations rely on today are reactive by design, blind to the wider network context, and either too rigid to handle a new attack or too risky to be trusted to act on their own. SwarmSec is built to directly address all three: it reasons about incidents instead of only matching signatures, it checks every incident against a live topology graph before deciding how serious it is, and it verifies every fix in a sandbox before touching production, which removes the need for a human to be present for every single response.

5. Objectives

The objectives of this project are as follows:

- **Build a multi-agent detection and response pipeline.** Design and implement a set of cooperating agents, each responsible for one stage of the incident lifecycle – monitoring, detection, context mapping, remediation, verification, and deployment – instead of one monolithic detection script.
- **Model the network as a live graph.** Use Neo4j to represent the network topology, so that any detected incident can be checked against the graph to estimate its blast radius before any action is taken.
- **Generate remediation dynamically.** Enable the remediation agent to write commands (firewall rules, container isolation commands, credential rotation calls) specific to the incident at hand, instead of relying on a fixed library of pre-written scripts.
- **Verify every fix before deployment.** Route every generated fix through an isolated Docker sandbox first, so that a faulty or unexpected fix is caught before it can affect the live environment.
- **Keep a complete audit trail.** Log every detection, decision, and action taken by the system in a structured, queryable form, so the full lifecycle of an incident can be reviewed after the fact.
- **Evaluate against a traditional baseline.** Replay the same simulated attacks against a standard firewall/Suricata setup and against SwarmSec, and compare detection time, response time, and whether either setup caused any disruption to unaffected services.

6. Scope of Work

This project is scoped as a working proof-of-concept, built and evaluated inside a controlled lab environment rather than a live production system. The environment consists of a small cluster of Docker containers standing in for application servers, a simulated internal network between them, and a set of scripted attack scenarios that are replayed against the system during testing.

Within this environment, the project covers the full pipeline described in the objectives above: monitoring, detection, graph-based context mapping, dynamic remediation, sandbox verification, deployment, and logging. It also covers building and populating the Neo4j topology graph for the test environment, and building the evaluation harness used to compare Swarm-Sec against a traditional baseline.

The following is explicitly out of scope for this project: securing physical hardware or on-premise networks, defending against attacks that require specialised hardware access (such as side-channel attacks), building a commercial-grade user interface or dashboard beyond what is needed to demonstrate and evaluate the system, and integrating with a real production cloud provider account. These are reasonable directions for future work but are not part of what will be delivered by the end of this project.

Four specific attack scenarios define the functional scope of the system:

- Multi-stage port scanning and lateral movement between containers.
- Layer-7 API abuse that stays under standard rate-limit thresholds.
- Malicious routing and IP/ARP spoofing inside the internal network.
- Unauthorized reverse-shell execution inside a production-like container.

A full technical walkthrough of how each scenario is detected and handled is given in Chapter 9.

7. Technology Stack

The technology choices for SwarmSec are grouped below by the layer of the system they belong to. Each tool was chosen for a specific reason rather than by default, which is explained alongside it.

Table 7.1: Technology stack used in SwarmSec

Layer	Technology	Reason for Use
Core Language	Python 3.11	Fast to prototype in, and has mature libraries for ML, log parsing, and API integration.
Agent Reasoning	LLM API (e.g., Claude/GPT)	Used for reading raw log data and reasoning about intent, instead of relying only on fixed rules.
Agent Orchestration	LangChain	Provides a ready-made way to chain tool calls, memory, and reasoning steps for each agent.
Graph Database	Neo4j + Cypher	Stores the live network topology and supports fast multi-hop “blast radius” queries.
Sandbox Testing	Docker + Docker Compose	Runs a generated fix in an isolated container to verify it before deployment.
Agent Communication	Redis Pub/Sub	Lightweight message bus used for agents to notify each other of new events.
Logging / Audit Store	MongoDB	Stores structured incident logs and the full audit trail as JSON-like documents.
Traffic Capture	Suricata / raw packet logs	Supplies the raw network events that the monitoring agents read from.
API Layer	FastAPI	Exposes agent status, logs, and manual override endpoints for the evaluation harness.
Version Control	Git + GitHub	Used for source control and collaboration between the two team members.

This stack was chosen to keep every layer replaceable – for example, Redis could later be swapped for a heavier message broker, or MongoDB for a different log store – without needing to redesign the agents themselves, since each agent only depends on a narrow interface to its layer rather than the technology directly.

8. System Design

8.1 Design Philosophy

SwarmSec is deliberately built as a set of small, narrowly-scoped agents rather than one large system that tries to detect, decide, and act all at once. This mirrors how a real security operations team is usually structured – one person watches dashboards, another investigates a flagged alert, another decides on and writes a fix, and a change-management process tests that fix before it goes live. Splitting the system this way makes each agent easier to test in isolation, easier to reason about when something goes wrong, and easier to replace or upgrade individually as the project develops.

8.2 Agent Roles

Monitoring Agents run continuously across every node in the lab environment, tailing logs, watching network traffic, and tracking container process activity. They do not make any judgement calls themselves – their only job is to forward events to the detection agent in a consistent format.

Detection Agent takes events from the monitoring agents and decides whether they look suspicious. It first runs a fast, rule-based pass (similar to a standard IDS) to catch obvious cases, and then, for anything ambiguous, uses an LLM to reason over the raw log content the way a human analyst would, looking for patterns that a fixed rule would miss.

Graph-Context Agent takes anything flagged by the detection agent and queries the Neo4j topology graph to find out what else in the network the affected node can reach. This produces a blast-radius estimate that is used to decide how urgent and how careful the response needs to be.

Remediation Agent uses the incident details and the blast-radius estimate to write a specific fix – for example, an `iptables` rule to block a source IP, a command to isolate a container from the network, or a call to rotate a compromised credential.

Sandbox Verification Agent takes the generated fix and runs it inside a temporary Docker container that mirrors the affected part of the environment, to confirm the fix behaves as expected and does not break anything else, before it is allowed anywhere near production.

Deployment and Logging Agent applies the verified fix to the live environment and writes a complete record of the incident – what was detected, what the graph context showed, what fix was generated, how it was verified, and when it was applied – into the MongoDB audit log.

8.3 Data Flow

1. A monitoring agent observes an event (e.g., a port scan) and publishes it to the message bus.
2. The detection agent picks up the event, scores it, and if it looks suspicious, passes it forward.
3. The graph-context agent queries Neo4j to determine which nodes are reachable from the affected container.
4. The remediation agent generates a fix based on the incident type and blast-radius result.

5. The sandbox verification agent tests the fix in an isolated container and reports back pass or fail.
6. If the fix passes, the deployment agent applies it to the live environment and logs the full incident; if it fails, the incident is escalated for human review instead of being applied blindly.

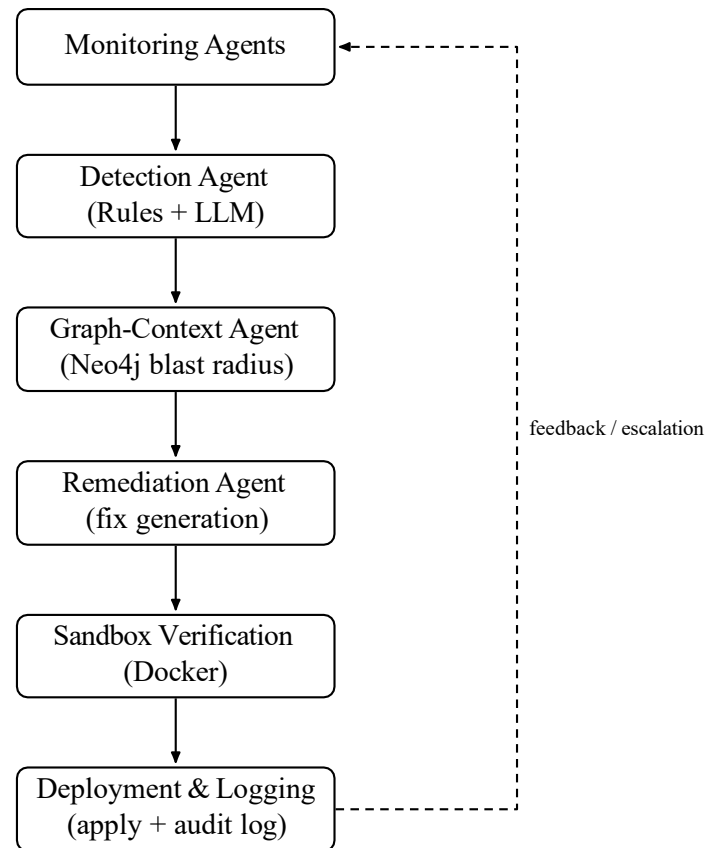


Figure 8.1: SwarmSec agent pipeline and data flow

8.4 Deployment View

For development and testing, all agents, Neo4j, Redis, and MongoDB run as separate Docker containers on a single host, connected through a private Docker network, with the sandbox verification agent given its own isolated sub-network so that a fix being tested can never accidentally reach the rest of the lab environment.

9. Methodology

The project follows an iterative, detect-verify-fix methodology, built up in stages rather than all at once.

Stage 1 – Environment setup. A lab environment is built using Docker containers standing in for application servers, connected through a simulated internal network. The Neo4j graph is populated with this topology so that the graph-context agent has real data to query from day one.

Stage 2 – Detection. Monitoring agents are connected to the log and traffic sources in the lab environment, and the detection agent is built and tuned first with rule-based checks, then extended with LLM-based reasoning for the cases rules alone cannot catch.

Stage 3 – Context mapping. The graph-context agent is connected to Neo4j and tested with Cypher queries that answer the core blast-radius question: given this compromised node, what else can it reach.

Stage 4 – Remediation and verification. The remediation agent is built to generate fixes for each of the four target scenarios, and the sandbox verification agent is built alongside it so that every fix is tested as soon as it exists, rather than verification being added as an afterthought.

Stage 5 – Deployment and logging. The deployment agent and MongoDB-based audit log are connected last, once fixes are reliably passing sandbox verification, so that only tested logic is ever wired up to affect the live environment.

Stage 6 – Evaluation. Once the full pipeline is working, all four attack scenarios are replayed twice: once against a baseline setup using standard firewall and Suricata rules, and once against SwarmSec. Detection time, response time, and any unintended disruption caused by the fix are recorded for both, and the results are compared directly.

10. Threat Scenarios and Comparison with Traditional Security Tools

10.1 Scenario 1: Multi-Stage Port Scanning and Lateral Movement

An attacker who has compromised one container scans internal ports and attempts SSH or API credential brute-forcing to move to another container. A traditional IDS may flag the scan itself but has no automatic way of isolating the compromised container or rotating the credentials that were targeted. SwarmSec's detection agent identifies the scanning pattern across nodes, the graph-context agent traces the blast radius from the origin node, and the remediation agent isolates the compromised container and rotates the affected credentials, all verified in the sandbox first.

10.2 Scenario 2: Layer-7 API Abuse

Low-and-slow API scraping or abuse is designed specifically to stay under standard volume-based rate limits, which means threshold-based tools miss it entirely. SwarmSec's detection agent uses anomaly scoring on request behaviour and header fingerprints rather than raw volume, and the remediation agent generates a custom ingress filtering rule targeted at the specific pattern observed, instead of a blanket rate-limit change that could affect legitimate traffic.

10.3 Scenario 3: Malicious Routing and IP/ARP Spoofing

An attacker advertises a false route or poisons ARP tables to redirect internal traffic toward a rogue destination. A traditional setup typically only notices this once something downstream fails. SwarmSec's graph-context agent continuously compares routing metrics against the expected Neo4j topology map, and as soon as a discrepancy appears, the remediation agent re-injects the correct routing costs and firewall rules to drop the spoofed packets.

10.4 Scenario 4: Container Reverse-Shell Execution

A remote code execution bug is exploited to open an unauthorized outbound connection from inside a production container. SwarmSec's monitoring agents detect the unexpected process creation, the remediation agent kills the rogue sub-process and captures forensic logs, and the container's security profile is updated – all without needing to restart the entire application, which is a common side effect of manual incident response today.

10.5 Comparison Summary

Table 10.1: SwarmSec vs. traditional security tools

Aspect	Traditional Tools	SwarmSec
Detection	Static signatures and thresholds	ML anomaly detection with LLM-based log reasoning
Response	Fixed, pre-written scripts	Fixes generated dynamically for the specific incident
Network Context	Logs viewed in isolation	Full topology awareness through Neo4j
Testing	Scripts run directly on production	Fixes tested in an isolated Docker sandbox first
Human Involvement	Manual review and action needed	Runs independently, keeps an audit log

11. Fault Tolerance and Reliability

Because SwarmSec is allowed to act on its own, the reliability of the agents themselves matters just as much as the accuracy of detection.

Sandbox-first execution. No fix is ever applied to the live environment without first passing a sandbox test. This is the single biggest safeguard in the system, since it means a badly-generated fix causes a failed test rather than a live outage.

Rollback support. Every applied fix is logged with enough detail – the exact command run and the state before it was applied – to be reversed manually or automatically if it later turns out to have been the wrong call.

Escalation on low confidence. If either the detection agent or the remediation agent is not confident about an incident or a fix, the incident is flagged for human review instead of being acted on blindly. This keeps a human in the loop for genuinely ambiguous cases while still automating the clear-cut ones.

Agent isolation. Each agent runs as a separate process connected only through the message bus, so a crash in one agent – for example, the remediation agent going down – does not take down monitoring or detection, and the affected agent can be restarted independently.

Auditability. Every decision made by every agent is logged with a timestamp and the reasoning behind it, so that even automated actions can be reviewed and explained after the fact, which is important both for debugging the system during development and for building trust in it later.

12. Work Plan

The project is planned over a 14-week timeline, split roughly evenly between the two team members, with Sarthak Pandit primarily leading the detection and graph-context work and Tushar Parihar primarily leading the remediation, sandbox, and evaluation work, while both contribute to system integration and testing throughout.

Table 12.1: Week-wise work plan

Week	Planned Work
Week 1–2	Literature review, finalising the four attack scenarios, and setting up the lab environment (Docker containers, simulated internal network).
Week 3–4	Setting up Neo4j and populating the topology graph; setting up Redis and MongoDB for messaging and logging.
Week 5–6	Building the monitoring agents and the rule-based part of the detection agent; testing against Scenario 1 (port scanning).
Week 7–8	Adding LLM-based reasoning to the detection agent; building the graph-context agent and blast-radius queries.
Week 9	Building the remediation agent and connecting it to the graph-context agent’s output.
Week 10	Building the sandbox verification agent and Docker-based test environment.
Week 11	Building the deployment and logging agent; wiring the full pipeline together end to end.
Week 12	Testing all four scenarios against SwarmSec end to end; fixing integration issues.
Week 13	Building the baseline (firewall + Suricata) setup and running the comparison evaluation.
Week 14	Final documentation, report writing, and preparing the project demonstration.

A short buffer is built into weeks 12 and 13 in case any earlier stage overruns, since integration between agents is expected to be the most unpredictable part of the timeline.

13. Expected Outcomes

By the end of this project, the following are expected to be delivered:

A working prototype. A functioning SwarmSec pipeline that can detect and safely resolve all four target attack scenarios inside the lab environment, end to end, without manual intervention for the clear-cut cases.

A populated graph-context layer. A working Neo4j topology graph for the test environment, with Cypher queries that correctly answer blast-radius questions for any node in the network.

A sandbox verification layer. A Docker-based testing step that reliably catches faulty fixes before they can be applied to the live environment, demonstrated with at least one deliberately-broken fix that the sandbox correctly rejects.

A complete audit trail. A MongoDB-backed log of every incident handled during testing, detailed enough that any action taken by the system can be explained and traced back to the event that triggered it.

A comparison report. A written comparison of SwarmSec against a traditional firewall/Suricata baseline on the same four scenarios, covering detection time, response time, and whether either setup caused any disruption to unaffected services.

Beyond the scope of this project, the same architecture could reasonably be extended to a wider set of attack scenarios, connected to a real cloud provider account instead of a simulated lab environment, and given a proper dashboard for security teams to monitor and override the agents in real time.

14. References

1. N. Petrovic et al., “CyberLLM: A Multi-Agent LLM Framework for Autonomous Detection and Guarded Response in Automotive Cybersecurity,” arXiv:2608.06651, 2026. [Online]. Available: <https://arxiv.org/abs/2608.06651>
2. Z. Liu, “Multi-Agent Collaboration in Incident Response with Large Language Models,” arXiv:2412.00652, 2024. [Online]. Available: <https://arxiv.org/abs/2412.00652>
3. Neo4j, “Cybersecurity – Attack Path Analysis & Blast Radius,” Neo4j Developer Resources. [Online]. Available: <https://neo4j.com/developer/industry-use-cases/cybersecurity/>
4. EURASIP Journal on Wireless Communications and Networking, “Graph database-based network security situation awareness data storage method,” 2018. [Online]. Available: <https://link.springer.com/article/10.1186/s13638-018-1309-9>
5. “Threat Modelling in Internet of Things (IoT) Environments Using Dynamic Attack Graphs,” arXiv:2310.01689, 2024. [Online]. Available: <https://arxiv.org/pdf/2310.01689>
6. Docker Inc., “Docker Documentation.” [Online]. Available: <https://docs.docker.com/>
7. Suricata, “Open Source IDS/IPS/NSM Engine.” [Online]. Available: <https://suricata.io/>
8. Neo4j, “Graph Data Platform.” [Online]. Available: <https://neo4j.com/>
9. LangChain, “LangChain Documentation.” [Online]. Available: <https://python.langchain.com/>
10. Redis, “Pub/Sub Documentation.” [Online]. Available: <https://redis.io/docs/latest/develop/interact/pubsub/>
11. MongoDB, “MongoDB Documentation.” [Online]. Available: <https://www.mongodb.com/docs/>
12. FastAPI, “FastAPI Documentation.” [Online]. Available: <https://fastapi.tiangolo.com/>